

Traccia Algoritmi - Esercizi 1B e 2B

Trascrizione pulita ricavata dagli screenshot forniti. Il PDF contiene solo i test degli esercizi, non le soluzioni.

Nota: gli screenshot originali erano inclinati e non perfettamente nitidi; il testo è comunque leggibile in modo sufficiente per ricostruire le consegne. Ho aggiunto in fondo anche le immagini originali come riferimento.

Esercizio 1 (B) - Albero Binario

Si considerino le seguenti funzioni che implementano un Albero Binario in cui la parte informativa di ogni nodo è un numero intero:

- `createTree()` che restituisce un albero vuoto;
- `insert(A, x)` che inserisce il nodo x nell'albero A ;
- `left(A)` che restituisce il sottoalbero sinistro di A ;
- `right(A)` che restituisce il sottoalbero destro di A ;
- `info(A)` che restituisce il valore associato al nodo radice di A ;
- `setInfo(A, x)` che modifica il valore associato alla radice di A impostandolo a x ;
- `isNull(A)` restituisce `True` se l'albero A è vuoto e `False` altrimenti.

1. Si realizzi una efficiente funzione `pathRange(A, z1, z2)` che, dato un Albero binario A e due interi $z1$ e $z2$ con $z1 < z2$, restituisce `True` se e solo se esiste in A un cammino $r = v_0, v_1, \dots, v_h = q$ che va dalla radice r di A a una foglia q di A , con $h \geq 1$, tale che valgano entrambe le seguenti proprietà:

- ogni valore $\text{info}(v_i)$, per $i = 0, \dots, h$, sia compreso nell'intervallo $[z1, z2]$ (estremi inclusi);
- la sequenza dei valori $\text{info}(v_0), \text{info}(v_1), \dots, \text{info}(v_h)$ sia non decrescente, cioè $\text{info}(v_i) \leq \text{info}(v_{i+1})$ per ogni $i = 0, \dots, h - 1$.

2. Si discuta, motivandola adeguatamente, la complessità temporale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca, rispettivamente, il caso migliore e il caso peggiore.

3. Si discuta, motivandola adeguatamente, la complessità spaziale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca, rispettivamente, il caso migliore e il caso peggiore.

4. Si discuta se, assumendo che A sia un Albero Binario di Ricerca, la complessità individuata ai punti precedenti cambierebbe, motivando la risposta.

Esercizio 2 (B) - Grafo non orientato e pesato

Siconsiderino le seguenti funzioni che implementano un grafo non orientato e pesato:

- `createGraph(m)`, che, dato un valore intero m in input, crea un grafo con m nodi identificati da numeri interi compresi in $[0, m - 1]$, senza archi;
- `insertEdge(G, x, y, w)`, che, dato un grafo G , una coppia di nodi x e y e un numero reale positivo w , aggiunge un arco tra x e y di peso w (se l'arco esiste già, ne aggiorna il peso);
- `deleteEdge(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , rimuove, se presente, l'arco tra x e y ;
- `deleteNode(G, x)`, che, dato un grafo G e un nodo x , rimuove il nodo x da G . Se il nodo x possiede archi incidenti oppure x non è presente nel grafo G , il metodo solleva un'eccezione;
- `existsEdge(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , restituisce `True` se esiste un arco tra x e y , `False` altrimenti;
- `getWeight(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , restituisce il peso di tale arco;
- `size(G)`, che, dato un grafo G , restituisce il numero di nodi del grafo;
- `nodes(G)`, che, dato un grafo G , restituisce una lista contenente gli identificatori dei nodi di G .

Si noti che, a differenza di altre tracce, l'API non fornisce una funzione `neighbors` per ottenere direttamente i vicini di un nodo: tale funzione, se necessaria, dovrà essere implementata a partire dalle primitive sopra elencate.

Una rete di impianti deve essere collegata tramite cavi in modo da minimizzare il costo complessivo dei cavi installati. Il grafo G rappresenta tutti i collegamenti tecnicamente realizzabili tra coppie di impianti, dove il peso di un arco è il costo di realizzazione del relativo cavo. Per ragioni infrastrutturali, però, ciascun impianto può sostenere al più d cavi direttamente collegati ad esso, indipendentemente dal loro costo.

1. Si realizzi una funzione `neighbors(G, x)` che, dato un grafo G e un nodo x , restituisca la lista delle coppie $[y, w]$ tali che esiste l'arco (x, y) di peso w , utilizzando esclusivamente le primitive elencate sopra.
2. Si realizzi una funzione `isConnected(G)` che, dato un grafo G non orientato, restituisca `True` se e solo se G è connesso, ovvero se esiste un cammino tra ogni coppia di nodi di G , `False` altrimenti.
3. Si realizzi una funzione `boundedDegreeNetwork(G, d)` che, dato un grafo G non orientato e pesato e un intero $d > 0$, utilizzi la funzione `isConnected` del punto precedente per verificare che G sia connesso (in caso contrario la funzione deve restituire `None`) e, in caso affermativo, determini un sottoinsieme aciclico di archi di G tale che ogni nodo sia estremo di al più d archi del sottoinsieme, applicando la seguente strategia: si esaminano gli archi di G in ordine di peso non decrescente e si aggiunge l'arco corrente al sottoinsieme se e solo se la sua aggiunta non crea un ciclo e non porta nessuno dei suoi due estremi a superare il limite d di archi incidenti già selezionati. La funzione deve restituire la lista degli archi selezionati, ciascuno rappresentato come tripla $[x, y, w]$.
Si noti che, per valori di d sufficientemente piccoli, il sottoinsieme di archi restituito potrebbe non collegare tutti i nodi di G tra loro anche quando G è connesso: la funzione deve comunque restituire il sottoinsieme costruito secondo la strategia descritta, senza segnalare alcuna eccezione in tal caso.
4. Si analizzi e si motivi la complessità temporale nel caso peggiore di `neighbors`, `isConnected` e `boundedDegreeNetwork`, specificando la struttura dati ausiliaria adottata.
5. Si analizzi e si motivi la complessità spaziale nel caso peggiore di `neighbors`, `isConnected` e `boundedDegreeNetwork`.